



Universitat
Oberta
de Catalunya

(DBPHP) Desarrollo back-end con PHP
PEC 4 - Trabajando con APIs: Laravel
(Explicaciones)

Marc Turu Roca

Profesor: Samuel González Rodríguez

Fecha de entrega: 21/01/2026

Universitat Oberta de Catalunya (UOC)

Tabla de contenidos

0. Introducción	2
A1. Instalación de Laravel	2
A2. Sistema de autenticación	3
A3. Crear las migraciones y modelos necesarios. Uso de Tinker.	4
A4. Generar contenidos ficticios	6
A5. Implementar el frontend de la aplicación	7
A6. API	9
A7. Publicación	10
A8. Opinión de los 3 entornos	12

0. Introducción

La PEC 4 consistía en desarrollar un sitio web para gestionar museos y exposiciones con una base de datos **MySQL** en PhpMyAdmin usando **Laravel**. El proyecto incluyó un directorio de museos y exposiciones, vistas variadas, autenticación de usuarios y acceso a datos en **JSON**. Se emplearon **Tailwind** y **Vite** para estilos, **Eloquent** para la base de datos, **Factory**, **Faker** y **Seeder** para contenidos, y **Breeze Blade** para las plantillas.

Se resumen las implementaciones de las actividades en el entorno local antes de probarlas en el servidor, siguiendo el flujo del desarrollo y reflejando la resolución de cada actividad, incluyendo correcciones posteriores cuando surgieron errores.

A1. Instalación de Laravel

Para el inicio de esta PEC, dentro del directorio `C:\wamp64\www` se ejecutó el comando `$ composer create-project laravel/laravel dbphppec4_museums` para descargar Laravel, crear la estructura completa y dejar el proyecto listo para usar.

Con la instalación inicial hecha, se pasó a la configuración de MySQL. Para ello, se editó el archivo `dbphppec4_museums/.env` y se configuró para el uso de la BD phpMyAdmin:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=dbphppec4_db
DB_USERNAME=root
DB_PASSWORD=
```

Para poder solucionar un error de cache y sesiones debido a que todavía no se habían creado las tablas, se cambiaron los siguientes parámetros de `dbphppec4_museums/.env`:

```
CACHE_STORE=file
SESSION_DRIVER=file
```

Y se limpió la configuración para que se tomaran los nuevos valores del archivo:

```
$ php artisan config:clear; $ php artisan cache:clear
```

Esta instalación se diferenció de la hecha con Drupal en la PEC 2 por la facilidad de despliegue de un proyecto propio, la configuración clara en `dbphppec4_museums/.env`, la creación automática (y más visual) de tablas en la BD MySQL como *users*, *cache*, *jobs*, *migrations*, etc y la orientación a arquitectura MVC de php en vez de CMS.

A2. Sistema de autenticación

Para esta segunda actividad, inicialmente se instaló la librería **Breeze** con composer:

```
$ composer require laravel/breeze --dev
```

 e instalamos el sistema de autenticación Breeze con Artisan con la opción blade: `php artisan breeze:install blade`, `npm install`, `npm run dev` y, en vez de ejecutar `php artisan migrate`, se eliminaron las tablas creadas e se hizo `php artisan migrate:fresh` para *resetear* la BD y para que se aplicaran correctamente las migraciones.

Aún así, surgió el siguiente error al intentar crear la tabla *users*, ya que Laravel definía la columna *email* como VARCHAR(255) y le asignaba un índice único:

```
SQLSTATE[42000]: Syntax error or access violation: 1071 Specified key was too long; max key length is 1000 bytes.
```

Para solucionarlo, se configuró Laravel para que estableciera una longitud por defecto de 191 caracteres en las columnas *string* que vayan a tener índice (como los *unique*). Esto se hizo modificando el archivo `App/Providers/AppServiceProvider.php` en el método `boot()` con:

```
use Illuminate\Support\Facades\Schema;
Schema::defaultStringLength(191);
```

Seguidamente, tras comprobar la creación automática de un menú en la parte superior de la página inicio con las opciones *Log in* y *Register*, a través de la opción *Register* de http://localhost/dbphppec4_museums/public/register, se dió de alta a un nuevo usuario de pruebas con las siguientes credenciales:

email:usuario:contraseña | admin@fakemail.com:admin:uoc-25-S1@

Para después hacer *Logout* y probar de hacer *Login* en http://localhost/dbphppec4_museums/public/login con este mismo usuario.

Más adelante, con un simple CTRL+F, se pudo encontrar el mensaje inicial de bienvenida (localizado en `dashboard.blade.php`) y se cambió el código a

```
<p>Nos alegramos de volverte a ver, {{ auth()->user()->name }}!</p>
```

para finalmente comprobar el correcto funcionamiento de la parte de *Profile* desde http://localhost/dbphppec4_museums/public/profile.

A3. Crear las migraciones y modelos necesarios. Uso de Tinker.

Para esta tercera actividad, se siguió un flujo de creación de las dos entidades **topic** y **museum**, definición de una relación muchos-a-muchos entre ambas mediante una pivot table (**museum-topic**), creación de las migraciones y adición de los atributos como se especificaba en la **R1**, y uso de **Tinker** para crear al menos dos museos reales y asignar mínimo dos temáticas a cada museo.

1. Modelo y migración para Topic. Se ejecutó `$ php artisan make:model Topic -m` para crear `app/Models/Topic.php` y `database/migrations/2026_01_19_205208_create_topics_table.php`.

Seguidamente, se editó la migración para que quedara de la siguiente manera:

```
Schema::create('topics', function (Blueprint $table) {  
    $table->id();  
    $table->string('name', 255);  
    $table->timestamps();  
});
```

2. Modelo y migración para Museum. Se ejecutó `$ php artisan make:model Museum -m` para crear `app/Models/Museum.php` y `database/migrations/2026_01_19_205916_create_museums_table.php`.

Seguidamente, se editó la migración para que quedara de la siguiente manera:

```
Schema::create('museums', function (Blueprint $table) {  
    $table->id();  
    $table->string('name', 255);  
    $table->string('city');  
    $table->text('schedule');  
    $table->enum('guided_tours', ['sí', 'no']);  
    $table->decimal('price', 8, 2);  
    $table->string('image');  
    $table->timestamps();  
});
```

3. Pivot table (Museum-topic). Debido a que las dos tablas formaban una relación muchos-a-muchos, en el diagrama se crea una nueva clase, por lo que en BD y código es necesario materializarla para poder expresar dicha relación. Para hacerlo, se ejecutó `$ php artisan make:migration create-museum-topic-table` para crear

```
database/migrations/2026_01_19_210329_create_museum_topic_table.ph
```

p. Seguidamente, se editó la migración para que quedara de la siguiente manera:

```
Schema::create('museum_topic', function (Blueprint $table) {
    $table->id();
    $table->foreignId('museum_id')->constrained()->onDelete('cascade');
    $table->foreignId('topic_id')->constrained()->onDelete('cascade');
});
```

4. Migraciones. Se ejecutó `$ php artisan migrate` y se crearon las tablas en phpMyAdmin.

5. Relaciones. En el modelo de cada clase, se definieron las relaciones para topics:

```
return $this->belongsToMany(Museum::class);
```

y para museums:

```
return $this->belongsToMany(Topic::class);
```

6. Contenido real con tinker. Después de entrar en Tinker con: `$ php artisan tinker` y hacer `> use App\Models\Museum; > use App\Models\Topic;`, se crearon cuatro temáticas con la estructura:

```
> $history = Topic::create(['name' => 'Historia del arte']); (+3)
```

dos museos reales con la estructura:

```
> $british = Museum::create([
    .   'name' => 'Museo Británico',
    .   'city' => 'Londres',
    .   'schedule' => '10:00-17:30',
    .   'guided_tours' => 'sí',
    .   'price' => 15.50,
    .   'image' => 'british.jpg'
    .   ]); (+1)
```

y se les añadieron dos temáticas a cada uno:

```
> $british->topics()->attach([$history->id, $archaeology->id]); (+1)
```

Finalmente se hizo un `> $british->update(['schedule' => 'Lunes a sábado, 10:00-20:00']); (+1)` para que la sintaxis del *schedule* de cada museo fuera más precisa.

A4. Generar contenidos ficticios

Para esta actividad, se generó el **Factory** para definir cómo se crearían los datos de prueba, **Faker** para generar contenido ficticio y **Seeder** para activar e insertar datos en BD:

1. `$ php artisan make:factory MuseumFactory --model=Museum` y su edición:

```
return [
    'name' => $this->faker->unique()->company . ' Museum',
    'city' => $this->faker->unique()->city,
    ...
];
```

3. `$ php artisan make:seeder TopicSeeder` y su edición, creando 4 categorías.

2. `$ php artisan make:seeder MuseumSeeder` y su edición:

```
$topics = Topic::all();
Museum::factory(40)->create()->each(function ($museum, $index) use ($topics) {
    if ($index < 20) {
        $museum->topics()->attach(
            $topics->random(rand(2, $topics->count()))->pluck('id')->toArray()
        );
    } else {
        $museum->topics()->attach(
            $topics->random(1)->pluck('id')->toArray()
        );
    }
});
```

4. Actualizar `DataBaseSeeder.php` para ejecutar todos los Seeders juntos:

5. `$ php artisan db:seed` para ejecutar los Seeders y añadir los datos nuevos a la BD.

Debido a que la generación de contenido de museos con Faker creaba horarios demasiado irreales, se decidió editarlos en Tinker con el siguiente script:

```
> $days = ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', ...];
> $museums = Museum::where('id', '>', 2)->get();
> foreach ($museums as $museum) {
    $startDay = rand(0, 2); $endDay = rand($startDay + 1, 6);
    $startHour = rand(8, 12); $endHour = rand($startHour + 1, 21);
    $startMinute = rand(0, 1) * 30; $endMinute = rand(0, 1) * 30;
    $schedule = $days[$startDay] . ' a ' . $days[$endDay] . ', '
    . sprintf('%02d:%02d', $startHour, $startMinute) . '-'
    . sprintf('%02d:%02d', $endHour, $endMinute);
    $museum->update(['schedule' => $schedule]);
}
```

A5. Implementar el frontend de la aplicación

Para esta actividad, a diferencia de la PEC 3, donde en un mismo archivo .php se controlaba la lógica (*backend* con PHP) y la vista (*frontend* con HTML y CSS), para implementar las funcionalidades, se usaron **controladores** a partir de los **modelos**:

\$ `php artisan make:controller MuseumController`, donde se obtuvieron los 5 museos iniciales para la página Home (1), el museo individual para la página Individual (3) y se devolvió la vista con los resultados de la consulta al final de cada función (2 y 4):

```
$fixedMuseums = Museum::whereIn('id', [1, 2])->get();
$randomMuseums = Museum::whereNotIn('id', [1, 2])
    ->inRandomOrder()->limit(3)->get();

return view('home', compact('fixedMuseums', 'randomMuseums'));
```

```
$museum = Museum::with('topics')->findOrFail($id);
return view('home', compact('museums'));
```

Aparte, también a diferencia de la PEC 3 donde las rutas utilizadas eran las de los propios archivos .php, esta vez se definieron las rutas web en el archivo `routes/web.php`

```
Route::get('/', [MuseumController::class, 'index'])->name('home');
Route::get('/museum/{id}', [MuseumController::class, 'show'])->name('museum.show');
```

Con la parte lógica creada, faltaba refinar la presentación de los contenidos en las vistas de `resources/views/home.blade.php` y `resources/views/museum.blade.php`, donde lo más destacable fue combinar PHP con HTML y usar `<x-app-layout>` con:

```
@foreach ($museums as $museum)
```

para mostrar los atributos necesarios de cada museo y, en la vista de museos individuales, utilizar de nuevo un bucle para mostrar todas las temáticas de ese museo.

Seguidamente, se observó que la barra de navegación estaba programada en `resources/views/layouts/navigation.blade.php`, concretamente en:

```
<div class="hidden space-x-8 sm:-my-px sm:ms-10 sm:flex">
    <x-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
        {{ __('Dashboard') }}
    </x-nav-link>
```

así que se añadieron los restantes **Home**, **API_museums**, **API_museum** y **API_topic** con el *helper url* además de controlar el comportamiento de la barra con `@guest` y `@auth` para saber qué mostrar dependiendo del estado del usuario en la web. **Siempre visibles**: Home, API_museums, API_museum y API_topic. **Visibles con @auth**: Dashboard, Profile y Logout. **Visibles con @guest**: Register y Login.

Se supuso que, como la propia utilización de Blade hace, en Register y Login se tenía que mostrar únicamente el formulario, ya que se trataba de un `<x-guest-layout>`. Si se quisiese implementar también más componentes como por ejemplo la barra de navegación, se podría sin problema añadiendo `<x-app-layout>` y ajustando los estilos.

También se observó el error de que todas imágenes a mostrar seguían el correcto patrón de enrutamiento `images/museums/b8befbf0-13c7-3d2b-a2ac-756093db0d2a.jpg`, menos las dos creadas manualmente con Tinker en la **A2**. Es por eso que se adaptaron sus rutas para que se pudieran obtener correctamente tanto en local como en el servidor con

```
name }}" width="200">
```

Finalmente, y como representa el encabezado de esta sección, se siguió mejorando el *frontend* para dar un aspecto visual más bonito y propio de una web en condiciones utilizando **Tailwind CSS** en los elementos de las vistas de cada página, sirviéndose en ocasiones de componentes ya creados, como por ejemplo `x-danger-button` o creando propios (combinando estilos Tailwind y normales, para que de esta manera, tanto si se compilaban y usaban los primeros como si no, la página web tuviera un aspecto similar).

Para que se aplicaran al completo, se tuvo que abrir otra terminal y ejecutar `$ npm run dev`. De esta manera, Laravel con Vite estaba en funcionamiento para desarrollo.

Más adelante, cuando se tuviera que compartir con producción, se ejecutaría `$ npm run build` para generar los archivos finales dentro de `public/build` y se eliminaría el archivo `public/hot`, el cual ignora `public/build` y siempre intenta cargar CSS/JS desde Vite (pero en el servidor no estará corriendo).

A6. API

Para esta actividad, se creó `$ php artisan make:controller Api/MuseumApiController`, donde se incluyeron tres métodos para controlar la funcionalidad de las APIs (`museums($page = 1)`, `museum($id)`, `topic($id, $page = 1)`). Estos métodos se declararon en un nuevo archivo `routes/api.php` (que ya incluía el prefijo `api` para los endpoints) y, para utilizar el *Middleware Laravel*, después de comprobar que `$ php artisan route:list` no contenía las URI api, se observó que no existía el archivo `RouteServiceProvider`, así que se creó con el siguiente cuerpo:

```
$this->routes(function () {  
    Route::middleware('api')  
        ->prefix('api')  
        ->group(base_path('routes/api.php'));  
  
    Route::middleware('web')  
        ->group(base_path('routes/web.php'));  
});
```

Después, se editó `config/app.php` para añadir los *providers* que faltaban, en especial:

```
App\Providers\AppServiceProvider::class,  
App\Providers\RouteServiceProvider::class,
```

Finalmente, se ejecutó `$ php artisan config:clear; $ php artisan route:clear; $ php artisan cache:clear` para resetear la configuración, rutas y caché.

Por lo que respecta al contenido de `MuseumApiController`, cada método `museums($page = 1)`, `museum($id)`, `topic($id, $page = 1)` obtenía todos los museos con sus temáticas, el museo específico con su(s) tematicas(s) o los museos de la temática, paginaba manualmente para evitar traer la *pivot table* y construía la respuesta paginada con los campos específicos a devolver (en *data*), ya fuesen todos los campos (para los dos primeros métodos) o solamente id, nombre y ciudad (para el tercer método). También se mostraba información extra (en *current_page*, *per_page*, *total* y *last_page*).

Si no se encontraba ningún resultado (ya fuese por el *id* y/o la *página*) se devolvía:

```
return response()->json(['message' => 'Not Found'], 404);
```

El *JSON result* de cada *API endpoint* se podía editar manualmente cambiando los valores de id y page pertinentes de la URL.

A7. Publicación

1. Primeramente, me dirigí a http://localhost/phpmyadmin/index.php?route=/database/export&db=dbphppec4_db y exporté la BD como SQL y, después de comprimirla, le cambié el nombre para que terminara en **.sql.zip**.
2. En segundo lugar, accedí al PhpMyAdmin del servidor remoto (utilizando las mismas credenciales que para acceder al servidor de pruebas) a través de la siguiente URL: <https://eimtcms2.techlab.uoc.edu/bbdd/>.
3. Debido a la falta de permisos para crear nuevas BD en el servidor, se eliminaron las tablas de la PEC 3 (ya que la PEC 2 se realizó con SQLite) de la BD **mturur** siguiendo el recurso <https://www.hostinger.com/support/4536290-how-to-delete-all-database-tables-in-phpmyadmin-in-hostinger/>. En teoría no haría falta porque los nombres de las tablas son diferentes, pero lo hice igualmente.
4. Importé la BD que exporté en el punto 1 (dentro de la BD ya creada **mturur**).
5. En el primer paso para el proyecto Laravel, para poder guardar los estilos a utilizar en el servidor, eliminé el archivo `public/hot` e hice `$ npm run build` para generar los estilos visuales finales.
6. En segundo lugar, borré la caché con `$ php artisan config:clear; $ php artisan cache:clear; $php artisan config:cache`.
7. Después, comprimir el proyecto de la PEC 4 que tenía en mi equipo (**C:\wamp64\www\dbphppec4_museums**).
8. Después de crear la carpeta **pec4/** en el servidor remoto, copié el .zip del proyecto en esa ubicación.
9. Accedí al servidor por conexión `ssh` en la terminal hasta mi carpeta de `/pec4`, descomprimí el .zip en cuestión¹ y se intentó eliminar la carpeta `/pec2`² manualmente (pero se cortaba el proceso) y mediante SSH (pero no tenía permisos), así que solo se pudo eliminar parcialmente, restando sin eliminar los `.../styles`.
Haciendo `$ mturur@eimtcms2:~$ du -sh /home/estudiants/mturur/public_html/pec2` se comprobó que solamente quedaban 6 MB de contenido.

¹ Este se eliminó posteriormente por motivos de seguridad.

² Para liberar espacio.

10. Dí permisos **664** a `.env` para poder editarlo a continuación y **777** a todo el contenido (subdirectorios incluidos) de `storage` y `bootstrap/cache`. Se hizo con la base de `$ chmod -R numeropermiso nombrecarpeta`.
11. Seguidamente, edité el archivo `.env` desde **FileZilla** para arreglar:

```
DBDATABASE=mturur
DBUSERNAME=mturur
DBPASSWORD=mi_contraseña_al_servidor

APP_URL=https://eimtcms2.techlab.uoc.edu/~mturur/pec4/dbphppec4_museums/public
```

12. Prácticamente acabando, al intentar acceder me salía un error 500 que se solucionó haciendo `$ php artisan optimize:clear`.
13. Una vez estos problemas estaban resueltos, probé que todas las funcionalidades de la PEC 4 funcionasen correctamente, como lo hacían en local.

A8. Opinión de los 3 entornos

Estas tres PECs han consistido en desarrollar un directorio de museos y exposiciones usando distintas tecnologías, cada una con ventajas e inconvenientes:

PEC 2. Drupal:

- + Interacción sencilla con la BD integrada (SQLite).
- + Contenidos bien organizados y reutilizables mediante *manage display*, vistas y componentes.
- + Buen control de roles y permisos sobre los contenidos.
- Limitaciones en la personalización de páginas y filtros.
- Problemas de muestra de contenido por la falta de autenticación.
- Configuración de la API compleja por módulos y formateo de campos.

PEC 3. PHP puro:

- + Control total y claro sobre la BD y su funcionamiento gracias a la creación manual.
- + Configuración sencilla de rutas API mediante *.htaccess*.
- + Estructura simple que facilitaba la personalización del *backend* y *frontend*.
- Gestión manual de la BD poco práctica.
- Menor reutilización de componentes al crearse todo desde cero.
- Gestiones de autenticación más limitadas.

PEC 4. Laravel:

- + Generación eficiente de datos de prueba con Factory, Faker y Seeder.
- + Reutilización de vistas y componentes con Breeze Blade.
- + Alta personalización visual del *frontend* con Tailwind CSS.
- Gestión de la BD menos visual al realizarse por código (Tinker entre otros).
- Posible confusión entre estilos Tailwind y CSS base.
- Configuración de la API compleja por middleware y dependencias.

Personalmente considero que las tres tecnologías tienen sus puntos fuertes y débiles.

Si tuviera que quedarme con algo de cada una, sería la **organización** visual de **Drupal**, la **gestión** de archivos y estructura de **PHP puro**, y la alta **personalización** de las funcionalidades de **Laravel** (ya sea de contenido como de estilos).